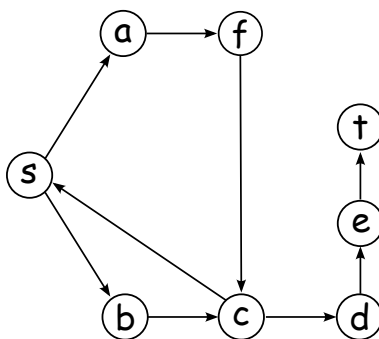


COMP364 Quiz 1

*Note: All search algorithms should use **graph** search (**not tree** search), unless stated otherwise. All outputs of search algorithms must be written using the format discussed in class.*

Question 1. BFS (10 points)

Trace the execution of the BFS algorithm on the graph below. You must write your output in the format discussed in class.

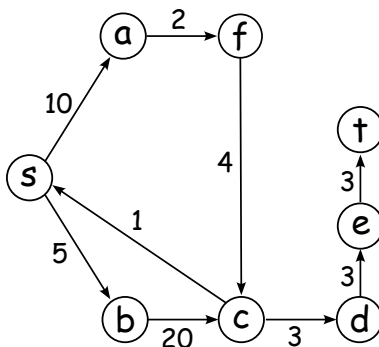


Question 2. DFS (10 points)

Trace the execution of the DFS algorithm on the same graph (above). You must write your output in the format discussed in class.

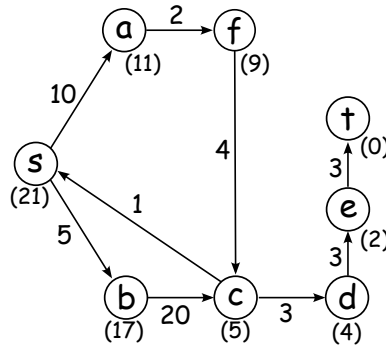
Question 3. Uniform cost search (also known as Dijkstra's algorithm) (10 points)

Trace the execution of the uniform cost search algorithm on the weighted graph below. You must write your output in the format discussed in class.

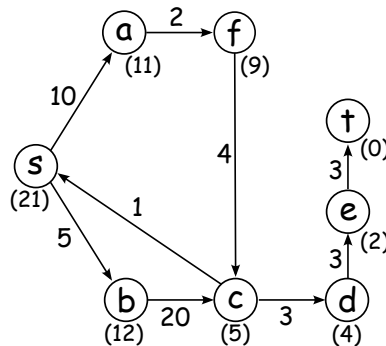


Question 4. A* search with consistent heuristic (10 points)

Trace the execution of the A* search algorithm on the weighted graph below. The heuristic value for each node is given below the node in parentheses. You must write your output in the format discussed in class.

**Question 5. A* search with admissible heuristic (10 points)**

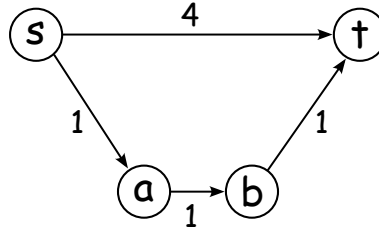
Trace the execution of the A* search algorithm on the weighted graph below, which is identical to the previous graph except for the heuristic value of b . You must write your output in the format discussed in class.

**Question 6. Admissible and consistent heuristics (5 points)**

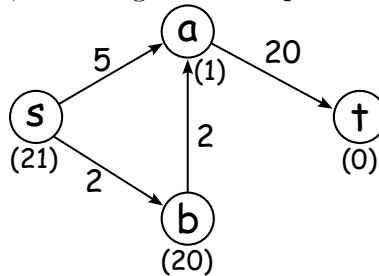
In the previous graph, which values for $h(b)$ make the heuristic (a) consistent and (b) admissible?

Question 7. Waiting for target expansion (10 points)

Trace the execution of the uniform cost search algorithm on the weighted graph below. Note that the algorithm does not stop when the target is first *discovered* (i.e., added to the frontier). The algorithm terminates when the target is *expanded* (i.e., removed from the frontier).

**Question 8. Re-expansion: tree search vs graph search (20 points)**

This question asks you to trace the execution of the A* search algorithm on the graph below, using two different variants: A* graph search and A* tree search. The heuristic given here is admissible but not consistent. One consequence of this is that A* *graph* search may fail to find the optimal path, whereas A* *tree* search is still guaranteed to find the optimal path. Why? It's possible that a node may need to be *re-expanded* if it gets added to the frontier with a lower priority than previously. In this example, node *a* gets re-expanded by tree search—but that re-expansion is missed by the graph search variant, resulting in a suboptimal path.



(a) **Graph search.** Trace the execution of the A* graph search algorithm on the graph above. Verify that the path found is suboptimal.

(b) **Tree search.** Trace the execution of the A* tree search algorithm on the graph above. Verify that the path found is suboptimal.

Question 9. NumPy and PyTorch vectors (10 points)

- Write NumPy code that creates a column vector called `x` with the elements 7.5, 2, and -4 .
- What is the output of `print(x.shape)`?
- Write PyTorch code that creates a tensor called `y` that is a 1D-shaped vector containing the elements 26 and 34.
- What is the output of `print(y.size())`?

Total number of points: **95**